# ImageSharp

ImageSharp is the high-performance part of the Six Labors stack you reach for when you need to load, inspect, process, and save images entirely in managed .NET code. It gives you one consistent image model whether you are building a thumbnail service, a photo workflow, a web upload pipeline, or a lower-level imaging tool.

This section is written as a guided set of articles rather than a flat feature list. Start with Getting Started if you are new to the library, then branch into loading, processing, formats, or lower-level pixel work as your needs get more specific.

The core model is: choose an image type, load or create pixels, run ordered processing operations, then save with an explicit encoder when output behavior matters. `Image` is convenient when you do not need direct pixel access, while `Image<TPixel>` makes the in-memory pixel format part of the type so high-performance row processing and format-specific work stay explicit.

For production code, the important choices are usually not individual method names. They are whether to identify before decoding, which pixel format to use in memory, how much metadata to preserve, which encoder settings define acceptable output, and when to customize configuration for formats, memory, or security.

# License

ImageSharp is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Projects that directly reference ImageSharp require a valid Six Labors license key at build time from version 3.2.0 in the previous major and 4.0.0 in the current major. Enforcement applies to direct dependencies only. This is enforcement of the existing license terms, not a license change.

# Install ImageSharp

ImageSharp is distributed on NuGet with preview and nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.ImageSharp -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

# How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it contains the complete license string required by the build.

> **IMPORTANT**
>
> Assembly-scoped community keys can be committed to a public repository. Contributors can use the included key to build the named assemblies without obtaining their own keys. Unrestricted community keys and commercial keys must remain private. Use environment variables or repository secrets for those keys.

To keep the file elsewhere, including a repository root above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <!-- Set the location of the supplied license file. -->
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

To use an environment variable instead, set this project property:

```
<PropertyGroup>
  <!-- Read the complete license string from the environment. -->
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

Set `SIXLABORS_LICENSE_KEY` to the complete contents of your issued license, not only its `Key` field. Use the original value without escaping. Replace the entire example value below with your issued license contents.

PowerShell:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
$env:SIXLABORS_LICENSE_KEY = 'Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

Bash:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
export SIXLABORS_LICENSE_KEY='Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

If you pass the license directly through `-p:SixLaborsLicenseKey`, escape every semicolon as `%253B`. MSBuild treats semicolons as property separators even inside a quoted argument. The pipe between assembly signatures stays unchanged.

PowerShell:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
dotnet publish "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

Bash:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
dotnet publish "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
```

You do not need to reference the licensing package directly. Projects that directly reference a Six Labors package carry its build-time validation.

# Community keys for public projects

For an open-source or source-available application, provide the exact `AssemblyName` of every project that directly references a Six Labors package, including test projects. Names are case-sensitive.

For example, if `MyProject.csproj` sets `<AssemblyName>MyProject.Core</AssemblyName>`, enter `MyProject.Core`. If it does not set `AssemblyName`, enter `MyProject`, the project filename without `.csproj`. If a shared props file sets the name, use that value. Do not include `.dll` or `.csproj`.

**Existing community license holders can request one early replacement with an assembly-scoped key. These upgrade requests are approved automatically, without another eligibility review. You do not need to wait for your current key to enter its renewal window.**

Apply at licensing.sixlabors.com. Select open-source or source-available, use the same email address, and provide the assembly names. Later requests follow the normal renewal window: the final 90 days before the expiry date in your license. Commercial licenses are not eligible for this early replacement.

# Security support

The latest major version of each library is eligible for security updates. Each major remains eligible for 12 months after the first stable release of its successor. A later major release does not shorten an existing support window.

Install the latest available patch or minor release within a supported major to receive security fixes. Older releases within that major are not maintained separately. This policy covers security fixes only. It does not include feature backports or extend Microsoft's support for the underlying .NET version. Security fixes remain at Six Labors' discretion.

# Start Here

- Getting Started walks through the core image types and the first end-to-end processing workflow.
- Loading, Identifying, and Saving covers file, stream, and buffer-based APIs plus encoder selection.
- Working with Metadata explains how to read and preserve EXIF, ICC, IPTC, XMP, and format-specific metadata.
- Color Profiles and Color Conversion covers ICC and CICP metadata, decode-time profile handling, and explicit working-space conversion.
- Image Formats explains format detection, encoders, decoders, and format registration.
- Processing Images introduces `Mutate()` and `Clone()` pipelines.
- Quantization, Palettes, and Dithering explains `Quantize()`, palette-based encoders, and dithering tradeoffs.
- Pixel Formats and Working with Pixel Buffers cover direct pixel access and advanced processing.

- Interop and Raw Memory covers `LoadPixelData(...)`, `WrapMemory(...)`, and contiguous-buffer interop.
- Configuration, Memory Management, and Security Considerations cover production-focused setup.
- Troubleshooting covers the common failure modes around format detection, streams, memory, and disposal.
- Migrating from System.Drawing maps common GDI-style workflows to ImageSharp APIs.
- Migrating from SkiaSharp maps common SkiaSharp image workflows to ImageSharp APIs.
- Recipes provides copy-pasteable solutions for common tasks.

# Implicit Usings

Set `UseImageSharp` in your project file to automatically import the most common ImageSharp namespaces:

```
<PropertyGroup>
  <UseImageSharp>true</UseImageSharp>
</PropertyGroup>
```

When enabled, ImageSharp adds implicit `global using` directives for:

- `SixLabors.ImageSharp`
- `SixLabors.ImageSharp.PixelFormats`
- `SixLabors.ImageSharp.Processing`

You can turn this off by removing the property or setting it to `false`.

# How to Use These Docs

- Start with loading, identifying, processing, and saving if you are new to ImageSharp.
- Move to formats, metadata, color profiles, and security before accepting untrusted images in production.
- Use pixel-buffer and interop pages when you need direct memory access rather than normal processors.
- Read the migration pages when replacing APIs whose image model differs from ImageSharp's typed pixel model.

# API Documentation

The API documentation is generated from the public source comments for the Six Labors libraries. Use it when you need exact type names, overloads, constructor signatures, enum values, default behavior, inherited members, and namespace-level navigation.

These reference pages are designed to sit alongside the article guides. Start with the articles when you are learning a feature or choosing an approach, then use the API reference when you need the precise member contract for implementation work.

The API reference covers the libraries documented on this site, including ImageSharp, ImageSharp.Drawing, ImageSharp.Web, Fonts, and PolygonClipper. Each product area is grouped by namespace so you can move from high-level entry points, such as image processing extensions or drawing canvas APIs, down to the supporting options, primitives, and model types.

When a reference page does not answer a design question, check the matching article section first. The source repositories remain available on GitHub for contributors and for cases where you need to inspect implementation details that are intentionally not part of the public API contract.